UNIVERSITÉ LAVAL

TP1 : annoter et convertir des identifiants

Avant de débuter

`org.Hs.eg.db` et `GO.db` sont des packages **Bioconductor** : ils sont déjà installés sur le serveur. Sur votre ordinateur : `BiocManager::install(c("org.Hs.eg.db", "GO.db"))`.

```
library(tidyverse)
library(org.Hs.eg.db)   # charge aussi AnnotationDbi
library(GO.db)
```

Lisez les messages de chargement : `AnnotationDbi` a sa propre fonction `select()`, qui **masque** celle de `dplyr`. Dans ce TP, écrivez toujours `dplyr::select()` pour choisir des colonnes et `AnnotationDbi::select()` pour convertir des identifiants.

Les données

Les résultats DESeq2 de l'expérience `airway` (TP3 du module de statistiques) : dexaméthasone contre non traité, plan `~ cell + dex`, `untrt` en référence.

Fichier	Contenu
`/home/public/enrichissement/airway_deseq2.tsv`	une ligne par gène : `gene` (identifiant Ensembl), `baseMean`, `log2FoldChange`, `stat`, `pvalue`, `padj`

Rappel : les fonctions

```
keytypes(org.Hs.eg.db)                    # les types d'identifiants qu'on peut donner
columns(org.Hs.eg.db)                     # ce qu'on peut obtenir
AnnotationDbi::select(org.Hs.eg.db, keys = ..., keytype = "...", columns = c("...", "...'
mapIds(org.Hs.eg.db, keys = ..., keytype = "...", column = "...")
```

À votre tour

0. Importez le fichier. Combien de gènes ? Combien ont été **testés** (`padj` non `NA`) ? Combien sont différentiellement exprimés (`padj` < 0.05 et |log2FoldChange| > 1), dont combien induits et combien réprimés ?
1. Listez les `keytypes()` et les `columns()` de `org.Hs.eg.db`. Avec `AnnotationDbi::metadata(org.Hs.eg.db)`, trouvez la date des sources Entrez (`EGSOURCEDATE`), GO, Ensembl et KEGG. Que remarquez-vous pour KEGG ?
2. Pour le gène *BRCA1* : son identifiant Entrez, son identifiant Ensembl, son nom complet (`GENENAME`) et sa position chromosomique (`MAP`). Combien d'accessions UniProt lui sont associées ?
3. Quels gènes se cachent derrière les identifiants Entrez `672`, `5468` et `7157` ? Donnez leurs symboles et leurs identifiants Ensembl.
4. Ajoutez à la table des gènes DE une colonne `symbole` avec `mapIds()` et `mutate()`. Combien de gènes DE n'ont pas de symbole ? Affichez les 10 gènes les plus **induits** avec leur symbole.
5. Faites la même conversion avec `AnnotationDbi::select()` sur les identifiants des gènes DE. Combien de lignes obtenez-vous ? Pourquoi pas autant que de gènes DE ? Trouvez les identifiants qui apparaissent plusieurs fois (`dplyr::count()`, `filter()`).
6. Avec `GO.db`, donnez le nom (`TERM`), la branche (`ONTOLOGY`) et la définition du terme `GO:0001838`. Quels sont les parents de `GO:0051384` (`GOBPPARENTS`) ?
7. Listez les annotations GO de *FKBP5* (colonnes `GO`, `ONTOLOGY`, `EVIDENCE`). Combien dans chaque branche ? Combien sont `IEA` ?

Et si… ?

Pour chaque question, **prédisez d'abord la réponse sans exécuter**, puis vérifiez dans R.

1. Et si les identifiants Ensembl portaient leur version (`ENSG00000152583.13`) ? Essayez `mapIds()` avec **tous** les identifiants versionnés (`paste0(de$gene, ".1")`), puis avec **un seul**.
2. Et si on écrivait `res |> select(gene, padj)` après `library(org.Hs.eg.db)` ? Lisez le message.
3. Et si on cherchait `"SEPT2"` comme `SYMBOL` ? Et comme `ALIAS` ? Pourquoi ce gène a-t-il changé de nom ?
4. Et si on utilisait `mapIds()` sur `ENSG00000004866` avec `multiVals = "first"`, puis `multiVals = "list"` ? Quelle information perd-on sans le savoir ?
5. Et si on joignait le tableau de `select()` (question 5) aux gènes DE avec `left_join()` ? Combien de lignes, et quel compte serait faux ensuite ?

Pour aller plus loin (avec Internet)

`biomaRt` interroge directement les serveurs d'Ensembl. Si votre poste a accès à Internet :

```
library(biomaRt)
ensembl <- useEnsembl(biomart = "genes", dataset = "hsapiens_gene_ensembl")
getBM(attributes = c("ensembl_gene_id", "hgnc_symbol", "entrezgene_id"),
      filters = "entrezgene_id", values = c(672, 5468, 7157), mart = ensembl)
```

Comparez avec votre réponse à la question 3. `listAttributes(ensembl)` et `listFilters(ensembl)` donnent tous les champs possibles.